

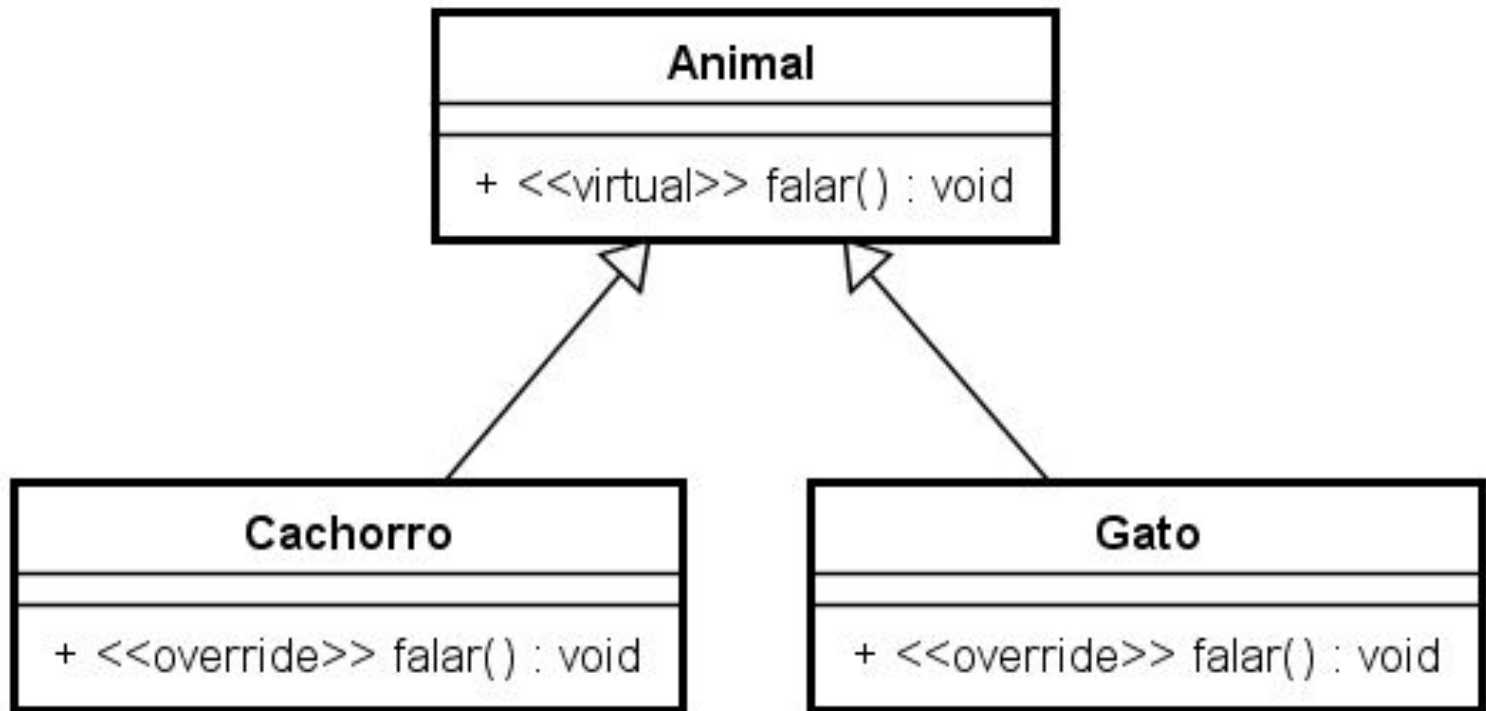


Sobrescrita de Métodos e Herança Múltipla

Prof. Dr. Valter Vieira de Camargo

POO - 2026/1

Analise ...



```

class Animal {
public:
    // Método virtual que pode ser sobrescrito
    virtual void falar() {
        std::cout << "Animal faz um som genérico.\n";
    }

    virtual ~Animal() {}
};

```

```

class Cachorro : public Animal {
public:
    void falar() override {
        std::cout << "Au au!\n";
    }
};

```

```

class Gato : public Animal {
public:
    void falar() override {
        std::cout << "Miau!\n";
    }
};

```

Veja este exemplo

- o método virtual tem corpo !
- fornece uma implementação **padrão**
- se uma classe filha não sobrescrever o método, ele terá o comportamento **padrão**
- se quiser forçar as filhas a sobrescreverem, use o virtual puro =0 (sem corpo)

```

int main() {
    Animal* a1 = new Cachorro();
    Animal* a2 = new Gato();

    a1->falar(); // Chama Cachorro::falar() -> "Au au!"
    a2->falar(); // Chama Gato::falar() -> "Miau!"

    delete a1;
    delete a2;

    return 0;
}

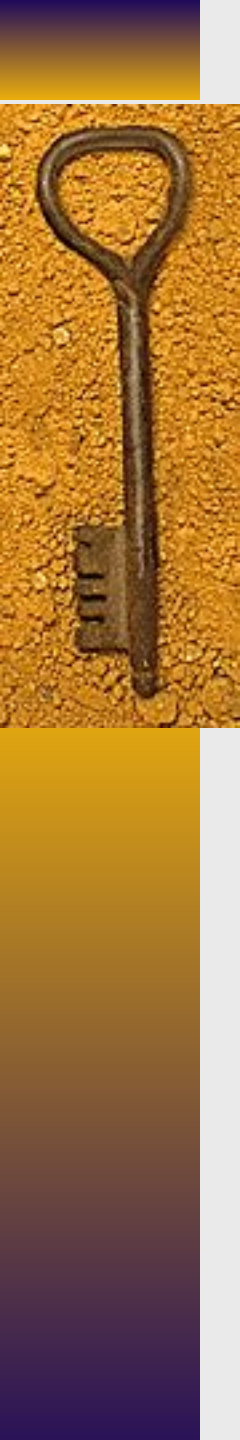
```



Herança Múltipla

Prof. Dr. Valter Vieira de Camargo

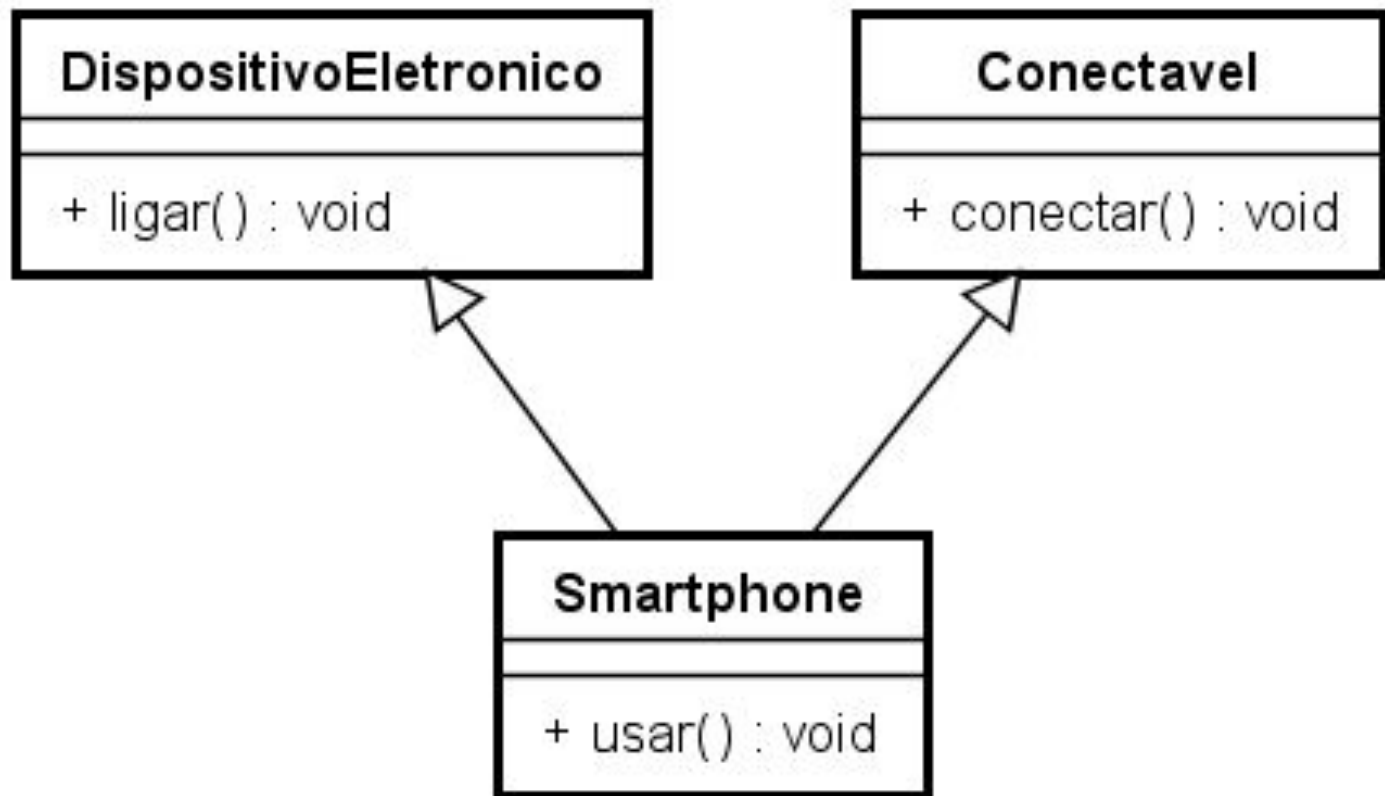
POO - 2025/1



Herança Múltipla

- ♦ Até o momento só vimos herança simples, em que classes filhas herdam de somente uma classe Base
- ♦ Mas classes filhas podem herdar de duas ou mais classes base, embora tenha que tomar cuidado com isso

Veja...



```
class DispositivoEletronico {
public:
    void ligar() {
        std::cout << "Dispositivo
            eletrônico ligado.\n";
    }
};
```

```
class Conectavel {
public:
    void conectar() {
        std::cout << "Dispositivo conectado à rede.\n";
    }
};
```

```
class Smartphone : public DispositivoEletronico, public Conectavel {
public:
    void usar() {
        std::cout << "Usando o smartphone.\n";
    }
};
```

```
int main() {

    Smartphone meuCelular;

    // Métodos herdados de ambas as classes base

    meuCelular.ligar();
    meuCelular.conectar();
    meuCelular.usar();

    return 0;
}
```

E se houver ambiguidade ? Classes base com métodos de mesmo nome ? Simples ! Resolva a ambiguidade !

```
class A {  
public:  
    void mostrar() { std::cout << "A\n"; }  
};
```

```
class B {  
public:  
    void mostrar() { std::cout << "B\n"; }  
};
```

```
class C : public A, public B {  
public:  
    void mostrar() {  
        A::mostrar();           // ou B::mostrar();  
    }  
};
```




Polimorfismo com Herança Múltipla

Prof. Dr. Valter Vieira de Camargo

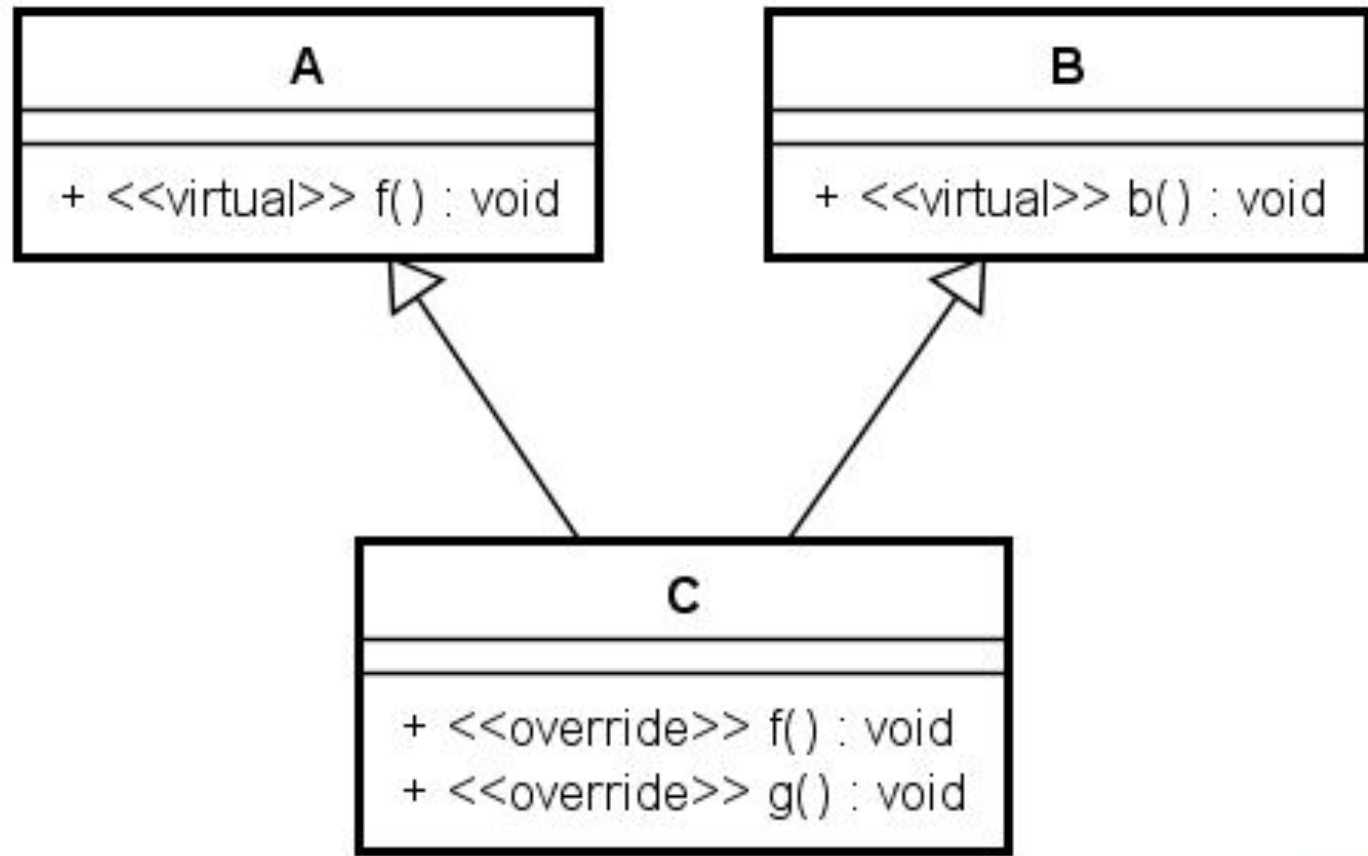
POO - 2026/1



Polimorfismo com Herança Múltipla

- ◆ Vamos analisar sob três ópticas:
 - Polimorfismo simples com herança múltipla
 - Problema do diamante - ambiguidade com herança de uma base comum
 - Solução: Herança Virtual

Polimorfismo Simples com H Múltipla



Polimorfismo Simples com H Múltipla

```
class A {  
public:  
    virtual void f() { std::cout << "A::f()\n"; }  
    virtual ~A() {}  
};  
  
class B {  
public:  
    virtual void g() { std::cout << "B::g()\n"; }  
    virtual ~B() {}  
};  
  
class C : public A, public B {  
public:  
    void f() override { std::cout << "C::f()\n"; }  
    void g() override { std::cout << "C::g()\n"; }  
};
```

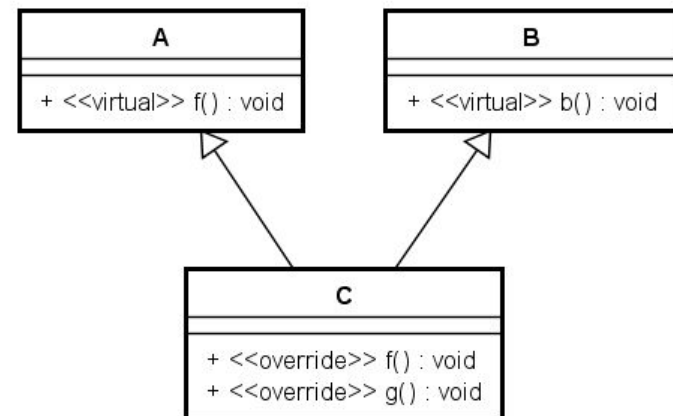
```
int main() {  
    C c;
```

c pode ser atribuído tanto a *pa* quanto a *pb*, pois *c* é um *A* e também é um *B*

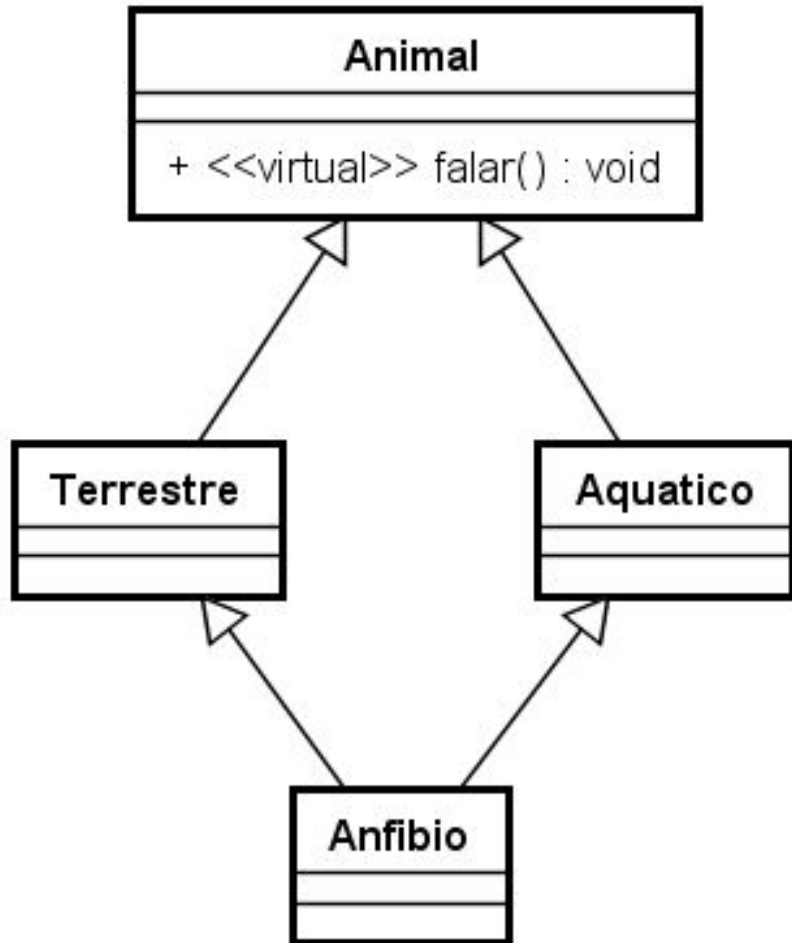
```
    A* pa = &c;  
    B* pb = &c;
```

```
    pa->f();    // C::f()  
    pb->g();    // C::g()
```

```
}
```



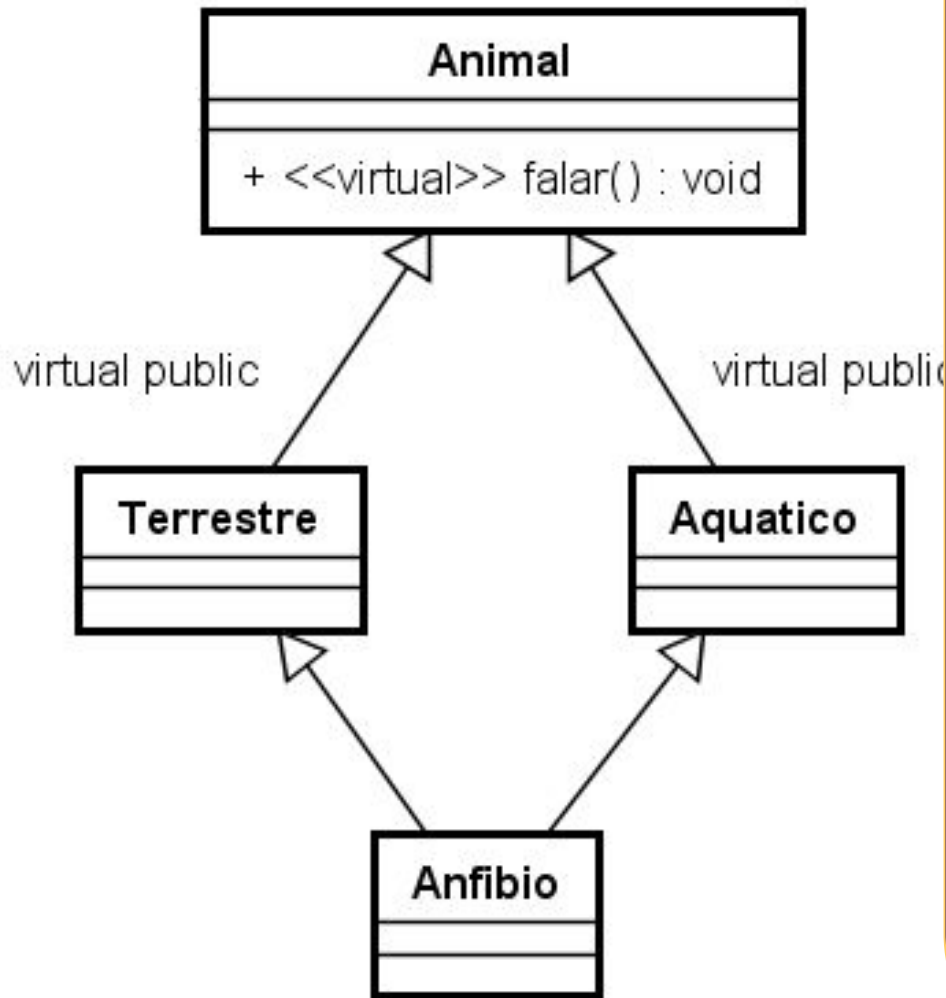
Problema do Diamante



Ambiguidade

- Anfibio tem duas copias da classe Animal, uma herdada por Terrestre e outra por Aquatico;
- no Main →
 Anfibio a;
 a.falar() → Ambiguidade
- Resolver isso com Herança Virtual

Problema do Diamante



Na herança simples, um objeto da classe Animal é criado para cada classe filha.

Então, se Anfibio chamar o “falar”, o compilador não sabe de qual você está chamando

Herança Virtual

- Garante que apenas uma cópia da classe Base exista
- Então, não haverá ambiguidade

```
int main() {
    Anfibio a;
    a.falar(); // Sem ambiguidade
}
```

// Classe base

```
class Animal {
```

```
public:
```

```
    Animal(const std::string& nome) : nome(nome) {  
        std::cout << "Animal construído: " << nome << std::endl;  
    }
```

```
    void mover() const {  
        std::cout << nome << " está se movendo." << std::endl;  
    }
```

```
protected:
```

```
    std::string nome;
```

```
};
```

/// Classe derivada de Animal com herança virtual

```
class Terrestre : virtual public Animal {  
public:  
    Terrestre(const std::string& nome)  
        : Animal(nome) {  
        std::cout << "Terrestre construído: " << nome << std::endl;  
    }  
  
    void andar() const {  
        std::cout << nome << " está andando na terra." << std::endl;  
    }  
};
```


// Classe derivada de Animal com herança virtual

```
class Aquatico : virtual public Animal {  
public:  
    Aquatico(const std::string& nome)  
        : Animal(nome) {  
        std::cout << "Aquatico construído: " << nome << std::endl;  
    }  
  
    void nadar() const {  
        std::cout << nome << " está nadando na água." <<  
std::endl;  
    }  
};
```

```
// Classe derivada de Terrestre e Aquatico
class Anfibio : public Terrestre, public Aquatico {
public:
    Anfibio(const std::string& nome)
        : Animal(nome), Terrestre(nome), Aquatico(nome) {
        std::cout << "Anfibio construído: " << nome << std::endl;
    }

    void viverEmAmbos() const {
        andar();
        nadar();
    }
};

int main() {
    Anfibio sapo("Sapo");

    sapo.mover();           // Método herdado de Animal
    sapo.viverEmAmbos();    // Métodos de Terrestre e
                             Aquatico

    return 0;
}
```



Exercício 1

Implemente uma hierarquia de classes com uma classe base `Mensagem` que possua um método virtual `enviar()`, com um comportamento padrão que apenas imprime "`Mensagem genérica enviada`".

Crie duas classes derivadas: `Email` e `SMS`. Cada uma deve sobrescrever `enviar()` com uma mensagem mais específica:

- `Email` → "`Email enviado`"
- `SMS` → "`SMS enviado`"

No `main`, crie um vetor de ponteiros para `Mensagem`, contendo instâncias de `Email` e `SMS`, e chame `enviar()` usando polimorfismo



Exercício 2

Crie duas classes base chamadas `Falante` e `Movimentador`. Cada uma deve ter um método virtual:

- `Falante::falar()` → imprime "Falando..."
- `Movimentador::mover()` → imprime "Movendo-se..."

Crie uma classe derivada `Robo` que herda de ambas e sobrescreve os dois métodos para imprimir mensagens específicas:

- `Robo::falar()` → "Robo falando"
- `Robo::mover()` → "Robo se movimentando"

No `main`, use ponteiros `Falante*` e `Movimentador*` para chamar os métodos corretos da instância de `Robo`.



Exercício 3

Implemente as seguintes classes:

- `Dispositivo` com um método virtual `ligar()`, que imprime "Dispositivo ligado".
- `Notebook` e `Tablet` herdam de `Dispositivo`.
- `Hibrido` herda de `Notebook` e `Tablet`.
 - Tente chamar `ligar()` em um objeto do tipo `Hibrido`.
 - Primeiro, implemente **sem herança virtual** e observe o erro de ambiguidade.
 - Depois, corrija usando **herança virtual**.



Exercício 4

Você está desenvolvendo um jogo de RPG tático. Nesse jogo, os personagens podem acumular **papéis múltiplos** durante a campanha. Os personagens podem ser Guerreiro, Mago e Guerreiro-Mago

Além dos papéis, cada personagem possui :

- Um nome
- Um nível
- Conjunto de atributos mágicos: força, inteligência e agilidade. Atributos esses que podem ser representados por valores inteiros
- Conjunto de itens.

Cada papel (classe de personagem) faz coisas **específicas**, por exemplo: Guerreiro só pode atacar, enquanto que Mago só pode lançar magia. Mas se o personagem for guerreiro e mago ao mesmo tempo ele pode executar as duas ações.

Veja um exemplo de como pode ser o Main →



Exercício 4

```
int main() {
    Atributos atributos = {15, 18, 10};
    GuerreiroMago arion("Arion", 12, atributos);

    arion.adicionarItem({"Poção de cura"});
    arion.adicionarItem({"Espada longa"});

    cout << "=== Informações do Personagem ===\n";
    arion.exibirInfo();

    cout << "\n=== Ações Disponíveis ===\n";
    arion.executarAcoes();

    // Demonstração de polimorfismo
    cout << "\n=== Ações via ponteiro para classe base ===\n";
    Personagem* p = &arion;
    p->executarAcoes();

    // pode continuar fazendo outros testes ..
    // por exemplo, instancie um personagem que é só
    // guerreiro e outro que é só Mago.

    return 0;
}
```

Dicas

- Implementem Atributos e Itens como Structs. Só agrupamento de dados.

